PLAN DE FORMATION

Deep Learning

Computer Vision & Traitement du Signal

Durée	**Niveau**
4 jours intensifs (28h)	Intermédiaire / Avancé

Format
Présentiel

Technologies : Python • PyTorch • torchvision • torchaudio • OpenCV • Hugging Face

1. Présentation Générale

Cette formation intensive de 4 jours couvre l'ensemble des architectures fondamentales et avancées du Deep Learning appliquées à la Computer Vision et au Traitement du Signal. Entièrement basée sur PyTorch, elle démarre par une initiation complète au framework avant de progresser des réseaux denses (MLP) jusqu'aux architectures Transformer (ViT), à travers des travaux pratiques intensifs sur des cas d'usage réels.

Objectifs Pédagogiques

- Maîtriser PyTorch : tenseurs, autograd, nn.Module, DataLoader, entraînement complet
- Implémenter les architectures DL : MLP, CNN, Transfer Learning, RNN/LSTM/GRU, Autoencoder, VAE, GAN, Diffusion Models, ViT
- Implémenter des solutions de Computer Vision : classification, détection, segmentation, débruitage
- Appliquer le Deep Learning au traitement du signal : classification de séries temporelles, débruitage de signaux
- Concevoir et entraîner des modèles génératifs (VAE, GAN, Diffusion Models) pour la synthèse et la reconstruction
- Exploiter le Transfer Learning et le Fine-tuning via torchvision et Hugging Face
- Déployer des pipelines ML complets de la préparation des données à l'inférence (TorchScript / ONNX)
- Appliquer les techniques d'Explainable AI (XAI) : Grad-CAM, SHAP, LIME, Attention Maps pour l'interprétabilité
- Comprendre et implémenter les modèles de diffusion (DDPM, Score-Based) pour la génération d'images de haute qualité

Prérequis

- Bases solides en Python (NumPy, Pandas, Matplotlib)
- Notions de Machine Learning classique (régression, classification, clustering)
- Mathématiques : algèbre linéaire, calcul différentiel, probabilités
- Aucune connaissance préalable de PyTorch requise — initiation incluse en Jour 1

Public Cible

- Chercheurs en vision par ordinateur et traitement du signal

Vue d'ensemble des Modules

Module	Architectures	Use Cases
Initiation PyTorch	Tenseurs, Autograd, nn.Module, DataLoader, Boucle d'entraînement	Premiers modèles, Pipeline complet

Module	Architectures	Use Cases
Fondamentaux MLP	Perceptron, Couches denses, Backpropagation	Classification de signaux 1D, Régression
CNN & Vision	ConvNet, VGG, ResNet, EfficientNet	Classification, Détection d'objets
Transfer Learning	Fine-tuning, Feature Extraction, torchvision models	Classification médicale
Séquences RNN	RNN, LSTM, GRU, Bidirectionnel	Séries temporelles, Signaux ECG/audio
Autoencoders & VAE	AE dense, ConvAE, VAE, Bottleneck	Débruitage, Compression, Anomalie
GAN	GAN, DCGAN, cGAN, Pix2Pix	Super-résolution, Synthèse, Augmentation
Vision Transformer	ViT, Swin, DeiT, SegFormer (HuggingFace)	Classification fine-grained, Segmentation sémantique
Explainable AI (XAI)	Grad-CAM, SHAP, LIME, Attention Maps, Feature Attribution	Interprétabilité CNN/ViT, Confiance diagnostique
Modèles de Diffusion	DDPM, Score-Based Models, Conditional Diffusion	Génération d'images, Débruitage, Inpainting

2. Jour 1 — Initiation PyTorch, MLP & CNN

Objectifs du Jour 1 Maîtriser les fondamentaux de PyTorch et implémenter MLP et CNN pour la Computer Vision et le Traitement du Signal.

- Maîtriser les fondamentaux de PyTorch : tenseurs, autograd, nn.Module, boucle d'entraînement
- Comprendre et implémenter les MLP pour la classification de signaux 1D
- Maîtriser les CNN et leurs variantes modernes (ResNet, EfficientNet) avec torchvision
- Appliquer le Transfer Learning et le Fine-tuning sur des datasets personnalisés

Session	Contenu & Activités
Initiation PyTorch — Fondamentaux	Tenseurs PyTorch vs NumPy, opérations de base, GPU/CPU (device), autograd et computational graph, nn.Module, optimiseurs torch.optim, premier modèle linéaire from scratch
TP PyTorch — Premiers pas	Manipulation de tenseurs, Dataset et DataLoader custom, boucle d'entraînement complète (train/val/test), sauvegarde et chargement de modèle (state_dict)
MLP & Fondamentaux DL	Neurone artificiel, fonctions d'activation (ReLU, Sigmoid, Softmax), forward/backpropagation, optimiseurs (SGD, Adam, RMSprop), régularisation (Dropout, BatchNorm) — tout en PyTorch
TP MLP — Signal 1D (PyTorch)	Classification de signaux ECG avec nn.Sequential, visualisation des gradients (retain_graph), courbes d'apprentissage avec Matplotlib, TensorBoard / W&B logging
CNN — Architectures & Théorie	Convolution 2D (nn.Conv2d), stride, padding, pooling, receptive field, ResNet (skip connections via nn.Module), EfficientNet, feature maps, Grad-CAM avec PyTorch hooks
TP CNN — Classification images (PyTorch)	Classification multi-classes avec torchvision datasets, transforms (Compose, Normalize, RandomAugment), DataLoader workers, métriques torchmetrics (F1, AUC-ROC)

Détail des Modules — Jour 1

Module 1.0 : Initiation à PyTorch

PyTorch est le framework de référence pour la recherche et l'industrie en Deep Learning. Son paradigme d'exécution dynamique (eager mode) et son API intuitive en font un outil idéal pour prototyper et déployer des modèles rapidement. Cette initiation couvre tous les blocs essentiels utilisés tout au long de la formation.

Tenseurs & Opérations :

- Création : torch.tensor(), torch.zeros(), torch.randn(), torch.arange()
- Opérations : broadcasting, reshape/view, squeeze/unsqueeze, cat/stack
- Transfert device : tensor.to('cuda'), tensor.cpu(), torch.device()
- Interopérabilité NumPy : tensor.numpy(), torch.from_numpy()

Autograd & Différentiation automatique :

- requires_grad=True, backward(), tensor.grad
- Computational graph dynamique (define-by-run)
- torch.no_grad() pour l'inférence (désactivation du gradient)
- Détachement du graphe : tensor.detach()

Construction de modèles avec nn.Module :

- Héritage de nn.Module, implémentation de __init__() et forward()
- nn.Linear, nn.Conv2d, nn.LSTM, nn.BatchNorm2d, nn.Dropout
- nn.Sequential pour des architectures simples
- model.parameters(), model.named_parameters(), model.eval() / model.train()

Pipeline d'entraînement complet :

- Dataset personnalisé : hériter de torch.utils.data.Dataset, __len__(), __getitem__()
- DataLoader : batch_size, shuffle, num_workers, pin_memory (GPU)
- Boucle d'entraînement : optimizer.zero_grad() → forward → loss.backward() → optimizer.step()
- Scheduler : torch.optim.lr_scheduler (StepLR, CosineAnnealingLR, OneCycleLR)
- Sauvegarde : torch.save(model.state_dict()), torch.load(), model.load_state_dict()
- Suivi d'expériences : TensorBoard (SummaryWriter) ou Weights & Biases

Exemple de code — Boucle d'entraînement PyTorch :

```
for epoch in range(num_epochs):
    model.train()  # mode entraînement
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        y_pred = model(X_batch.to(device))
        loss = criterion(y_pred, y_batch.to(device))
        loss.backward()
        optimizer.step()
```

Module 1.1 : MLP pour la Vision et le Signal (PyTorch)

Les Multi-Layer Perceptrons sont implémentés en PyTorch via nn.Linear et nn.Sequential. Ils constituent le bloc fondamental avant de passer aux architectures convolutives et récurrentes.

Contenu théorique & PyTorch :

- Architecture avec nn.Linear(in, out), fonctions d'activation nn.ReLU(), nn.Sigmoid(), nn.GELU()
- Loss functions : nn.CrossEntropyLoss(), nn.MSELoss(), nn.BCEWithLogitsLoss()
- Optimiseurs : torch.optim.SGD(momentum=0.9), torch.optim.Adam(weight_decay=1e-4)
- Régularisation : nn.Dropout(p=0.3), nn.BatchNorm1d(), EarlyStopping manuel
- Initialisation : torch.nn.init.kaiming_normal_() (He), xavier_uniform_()

Use Cases abordés :

- Classification de signaux ECG (5 classes : Normal, AFib, Bloc, Tachycardie, Autre)
- Régression sur données vibratoires de machines tournantes
- Détection d'anomalies dans des séries temporelles industrielles

Module 1.2 : CNN & Computer Vision (PyTorch + torchvision)

PyTorch via torchvision offre un accès direct aux architectures CNN pré-entraînées et à des utilitaires de traitement d'images. La bibliothèque torchvision.transforms facilite la construction de pipelines d'augmentation robustes.

API PyTorch clés :

- nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)
- nn.MaxPool2d(), nn.AdaptiveAvgPool2d() pour la flexibilité de l'entrée
- torchvision.models : resnet50(pretrained=True), efficientnet_b0(), vgg16()
- torchvision.transforms.v2 : RandomHorizontalFlip, ColorJitter, Normalize, CutMix, MixUp
- Hooks PyTorch pour Grad-CAM : register_forward_hook(), register_backward_hook()

Use Cases abordés :

- Classification binaire : images médicales (tumeur / sain) — torchvision.datasets.ImageFolder
- Classification multi-classes : reconnaissance de défauts industriels
- Visualisation Grad-CAM pour l'interprétabilité des décisions du modèle

3. Jour 2 — Transfer Learning, Séquences & Explainability (XAI)

Objectifs du Jour 2 Exploiter le Transfer Learning, maîtriser les architectures récurrentes (RNN/LSTM/GRU) pour le traitement du signal, et appliquer les techniques d'Explainable AI (XAI) pour l'interprétabilité des modèles.

- Maîtriser les architectures RNN, LSTM et GRU pour le traitement du signal et des séquences
- Concevoir et entraîner des Autoencoders pour la compression, le débruitage et la détection d'anomalies
- Comprendre les Variational Autoencoders (VAE) et leur espace latent structuré

Session	Contenu & Activités
Transfer Learning & Fine-Tuning (torchvision & Hugging Face)	torchvision.models pré-entraînés, gel des couches (requires_grad), remplacement de la tête, learning rate différentiel, timm library, OneCycleLR. Use cases : classification médicale, défauts industriels.
TP Transfer Learning — Fine-Tuning (PyTorch)	Fine-tuning ResNet50 sur Chest X-Ray, EfficientNet sur défauts industriels, domain adaptation ImageNet → satellite, benchmark feature extraction vs fine-tuning complet.
RNN, LSTM & GRU — Théorie PyTorch	RNN vanille et gradient vanishing, portes LSTM (nn.LSTM), GRU (nn.GRU), bidirectionnel, pack_padded_sequence, gradient clipping.
TP Signal — RNN/LSTM/GRU (PyTorch)	Classification de gestes (accéléromètre) avec nn.LSTM, débruitage ECG avec BiLSTM, prédiction de pannes machines, classification de phonèmes audio.

Session	Contenu & Activités
Explainable AI (XAI) — Interprétabilité	Grad-CAM, SHAP, LIME, Integrated Gradients, Attention Maps. Bibliothèques : Captum, pytorch-grad-cam, shap. Interprétabilité CNN et ViT.
TP XAI — Interprétabilité en Pratique (PyTorch)	Grad-CAM sur CNN médical (Captum), SHAP sur features tabulaires, LIME sur classification d'images, Attention Rollout sur ViT, rapport d'interprétabilité.

Détail des Modules — Jour 2

Module 2.1 : Transfer Learning (torchvision & Hugging Face)

PyTorch facilite le Transfer Learning grâce à torchvision.models et à l'écosystème Hugging Face (timm library). Le gel et dégel sélectif des couches se fait nativement via requires_grad.

Techniques PyTorch :

- Feature Extraction : for param in model.parameters(): param.requires_grad = False
- Remplacement de la tête : model.fc = nn.Linear(2048, num_classes)
- Learning rate différentiel : [{'params': base.parameters(), 'lr': 1e-5}, {'params': head.parameters(), 'lr': 1e-3}]
- timm library : timm.create_model('efficientnet_b4', pretrained=True, num_classes=N)
- Scheduler OneCycleLR : convergence rapide en fine-tuning

Use Cases :

- Classification médicale avec ResNet50 pré-entraîné ImageNet (Chest X-Ray, rétinopathie)
- Détection de défauts industriels avec EfficientNet fine-tuné
- Domain adaptation : modèle ImageNet → images satellite

Module 2.2 : RNN, LSTM & GRU (PyTorch)

PyTorch propose des implémentations optimisées des couches récurrentes (nn.RNN, nn.LSTM, nn.GRU) avec support natif du bidirectionnel et des séquences de longueurs variables via pack_padded_sequence.

API PyTorch clés :

- nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
- nn.LSTM(input_size, hidden_size, num_layers, bidirectional=True, dropout=0.2)
- nn.GRU(input_size, hidden_size) — alternative légère au LSTM
- pack_padded_sequence() et pad_packed_sequence() pour séquences variables
- Gradient clipping : torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

Use Cases Signal :

- Classification de gestes à partir d'accéléromètre 3 axes (UCI HAR Dataset)
- Débruitage de signaux ECG avec reconstruction temporelle BiLSTM
- Prédiction de pannes machines (vibrations, températures)
- Classification de phonèmes audio (spectrogramme STFT + LSTM)

Module 2.3 : Explainable AI (XAI) — Interprétabilité des Modèles Deep Learning

L'Explainable AI (XAI) regroupe les techniques permettant de comprendre et d'interpréter les décisions des modèles de Deep Learning. En contexte médical, industriel ou critique, l'interprétabilité est essentielle pour la confiance, la conformité réglementaire et le débogage des modèles. PyTorch offre un écosystème riche pour implémenter ces méthodes.

Techniques d'Attribution & Saliency Maps :

- Grad-CAM & Grad-CAM++ : visualisation des régions activées via gradients de la dernière couche convolutive
- Saliency Maps : gradient de la sortie par rapport à l'image d'entrée (vanilla gradient, SmoothGrad)
- Integrated Gradients : attribution axiomatic le long du chemin baseline → input (captum library)
- Layer-wise Relevance Propagation (LRP) : propagation inverse de la pertinence couche par couche
- Occlusion Sensitivity : masquage systématique de patches pour identifier les zones critiques

Méthodes Model-Agnostiques :

- SHAP (SHapley Additive exPlanations) : valeurs de Shapley pour quantifier la contribution de chaque feature
- LIME (Local Interpretable Model-agnostic Explanations) : approximation locale par modèle linéaire interprétable
- Anchors : règles de décision locales suffisantes pour expliquer une prédiction
- Counterfactual Explanations : identification des modifications minimales pour changer la prédiction

XAI pour Vision Transformers :

- Attention Rollout : agrégation des attention maps multi-couches pour visualisation globale
- DINO Attention Maps : interprétabilité self-supervised via les attention heads de ViT

- GradSAM : combinaison Grad-CAM + Segment Anything pour explications segmentées
- Transformer Attribution : méthodes spécifiques pour expliquer les décisions des ViT (Chefer et al.)

API PyTorch & Outils XAI :

- Captum (PyTorch) : bibliothèque officielle d'interprétabilité — IntegratedGradients, DeepLift, GradientShap, Occlusion
- pytorch-grad-cam : implémentation Grad-CAM, HiResCAM, EigenCAM, FullGrad pour CNNs et ViTs
- SHAP library : shap.DeepExplainer(model, background), shap.GradientExplainer pour modèles PyTorch
- LIME : lime.lime_image.LimeImageExplainer() pour expliquer les classifications d'images
- Hooks PyTorch : register_forward_hook() et register_backward_hook() pour capturer activations et gradients

Use Cases XAI :

- Diagnostic médical assisté : Grad-CAM sur CNN pour localiser les lésions sur radiographies thoraciques
- Contrôle qualité industriel : SHAP pour identifier les features discriminantes dans la détection de défauts
- Conformité réglementaire : rapport d'interprétabilité LIME/SHAP pour audit de modèles (IA Act, FDA)
- Debugging de modèles : détection de biais et de raccourcis (shortcut learning) via saliency maps
- Comparaison CNN vs ViT : analyse des stratégies d'attention et des zones de focus par architecture

4. Jour 3 — Autoencoders, VAE & Modèles de Diffusion

Objectifs du Jour 3 Maîtriser les Autoencoders (AE, ConvAE) et VAE pour le débruitage, la compression et la détection d'anomalies. Comprendre et implémenter les modèles de diffusion (DDPM) pour la génération d'images de haute qualité.

- Maîtriser les architectures GAN et leurs applications en Computer Vision
- Comprendre et implémenter les Vision Transformers (ViT) pour la classification et la segmentation
- Réaliser un projet fil rouge intégrant plusieurs architectures étudiées

Session	Contenu & Activités
Autoencoders — Fondamentaux PyTorch	Architecture encodeur-décodeur avec nn.Module, loss de reconstruction MSE, ConvAE avec nn.ConvTranspose2d, Denoising AE, skip connections (U-Net style).
TP Autoencoder — Débruitage (PyTorch)	Denoising ConvAE sur images médicales (IRM, CT), AE dense pour détection d'anomalies de séries temporelles, compression d'images satellites, clustering latent + UMAP.
VAE — Variational Autoencoder (PyTorch)	Reparametrization trick (mu + eps * std), KL Divergence, ELBO loss, beta-VAE pour disentanglement. Conditional VAE, interpolation latente, génération ECG synthétique.

Modèles de Diffusion — DDPM & Score-Based (PyTorch)	Forward/reverse process, noise schedule, U-Net conditionné par timestep, DDPM training loop, DDIM sampling accéléré. Bibliothèque diffusers (Hugging Face).

Détail des Modules — Jour 3

Module 3.1 : Autoencoders (AE & ConvAE) en PyTorch

En PyTorch, les Autoencoders sont implémentés via nn.Module avec une architecture encodeur-décodeur. Le décodeur convolutif utilise nn.ConvTranspose2d (transposed convolution) ou nn.Upsample + nn.Conv2d pour la reconstruction spatiale.

Architectures PyTorch :

- AE Dense : nn.Linear + nn.ReLU, bottleneck, reconstruction MSE
- ConvAE : encodeur nn.Conv2d + décodeur nn.ConvTranspose2d
- Deep AE avec skip connections (U-Net style) pour préserver les détails fins
- Denoising AE (DAE) : ajout de bruit (torch.randn_like()) en entrée

Use Cases :

- Débruitage d'images IRM et CT-scan (bruit Gaussien, bruit Poisson, artefacts)
- Compression d'images satellites
- Détection d'anomalies par seuil sur l'erreur de reconstruction MSE
- Clustering non supervisé via représentation latente + UMAP/t-SNE

Module 3.2 : Variational Autoencoder (VAE) en PyTorch

Le VAE en PyTorch se distingue par l'implémentation du reparametrization trick (z = mu + eps * std avec torch.randn_like(std)) qui rend l'échantillonnage différentiable et permet la rétropropagation.

Implémentation PyTorch :

- Encodeur : sortie de deux têtes linéaires mu et log_var
- Reparametrization : z = mu + torch.exp(0.5 * log_var) * torch.randn_like(log_var)
- KL Divergence : -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
- ELBO loss : loss = recon_loss + beta * kl_loss (beta-VAE pour disentanglement)

Applications avancées :

- Génération contrôlée avec Conditional VAE (label embedding concaténé au latent)
- Interpolation dans l'espace latent (morphing d'images via lerp / slerp)
- VAE pour la génération de signaux ECG synthétiques (augmentation de données)
- Détection d'anomalies : score ELBO pour détecter les OOD samples

Module 3.3 : Modèles de Diffusion (DDPM & Score-Based Models)

Les modèles de diffusion représentent l'état de l'art en génération d'images, surpassant les GAN en qualité et diversité. Le principe repose sur un processus de bruitage progressif (forward process) suivi d'un apprentissage du débruitage inverse (reverse process). En PyTorch, l'implémentation repose sur un U-Net conditionné par le pas de temps, entraîné à prédire le bruit ajouté.

Fondements Théoriques :

- Forward Process : ajout progressif de bruit gaussien $q(x_t \mid x_{t-1}) = N(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$
- Reverse Process : réseau neuronal $p_\theta(x_{t-1} \mid x_t)$ apprend à inverser le bruitage pas à pas
- DDPM (Denoising Diffusion Probabilistic Models) : Ho et al. 2020, loss = $MSE(\varepsilon, \varepsilon_\theta(x_t, t))$
- Score-Based Models : estimation du score $\nabla_x \log p(x)$ via score matching (Song & Ermon)
- Noise Schedule : linéaire ($\beta_1 \rightarrow \beta_T$), cosine schedule pour meilleure qualité
- Sampling : DDPM (lent, T steps), DDIM (rapide, déterministe, S << T steps)

Architecture U-Net pour Diffusion :

- U-Net avec skip connections : encodeur-décodeur convolutif avec concaténation des features
- Time Embedding : sinusoidal positional encoding du timestep t, injecté via addition ou FiLM
- Attention Blocks : nn.MultiheadAttention dans les résolutions intermédiaires du U-Net
- GroupNorm + SiLU : normalisation et activation standard pour les diffusion models
- Conditional Diffusion : injection de labels/texte via cross-attention ou concatenation

Implémentation PyTorch :

- Noise Scheduler : calcul des α_t, $\bar{\alpha}_t$, β_t pour le forward process avec torch.linspace()
- Training Loop : $x_0 \rightarrow$ sample $t \sim U(1,T) \rightarrow x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon \rightarrow$ loss = $MSE(\varepsilon, model(x_t, t))$
- Sampling Loop : $x_T \sim N(0,I) \rightarrow$ for t in reversed(range(T)): x_{t-1} = denoise(x_t, t)
- DDIM Sampler : accélération ×10-50 du sampling sans réentraînement
- Classifier-Free Guidance : $\varepsilon_{guided} = \varepsilon_{uncond} + w * (\varepsilon_{cond} - \varepsilon_{uncond})$ pour génération conditionnelle
- diffusers (Hugging Face) : DDPMPipeline, DDPMScheduler, UNet2DModel pour prototypage rapide

Variantes & Extensions :

- Latent Diffusion Models (LDM) : diffusion dans l'espace latent d'un VAE pré-entraîné (Stable Diffusion)
- Conditional Diffusion : class-conditional, text-conditional (CLIP embeddings), image-conditional
- Inpainting & Editing : masquage partiel pour complétion d'images guidée par diffusion
- Super-Resolution : diffusion conditionnée sur une image basse résolution (SR3, Palette)
- Diffusion pour le signal : génération et débruitage de séries temporelles et spectrogrammes

Use Cases Modèles de Diffusion :

- Génération d'images médicales : synthèse d'IRM/CT réalistes pour augmentation de données
- Super-résolution médicale : amélioration de la résolution d'images cliniques basse dose

- Inpainting & restauration : complétion de régions manquantes dans des images satellites ou médicales
- Débruitage d'images : alternative aux autoencoders avec qualité supérieure (score-based denoising)
- Génération conditionnelle : création d'images à partir de descriptions textuelles ou de labels
- Comparaison GAN vs Diffusion : trade-offs vitesse/qualité/diversité/stabilité d'entraînement

5. Jour 4 — GAN, Vision Transformers & Projet Final

Objectifs du Jour 4 Maîtriser les architectures GAN et leurs variantes, les Vision Transformers (ViT, Swin, SegFormer), et réaliser un projet fil rouge intégrant l'ensemble des compétences acquises.

- Maîtriser les architectures GAN et leurs applications en Computer Vision
- Comprendre et implémenter les Vision Transformers (ViT) pour la classification et la segmentation
- Réaliser un projet fil rouge intégrant plusieurs architectures étudiées

Session	Contenu & Activités
GAN — Théorie & Architectures PyTorch	Boucle d'entraînement adversariale PyTorch (update discriminateur puis générateur), GAN loss (BCEWithLogitsLoss, Wasserstein), DCGAN avec nn.ConvTranspose2d, cGAN avec label embedding, Pix2Pix avec patchGAN discriminator
TP GAN — Super-résolution (PyTorch)	Implémentation DCGAN sur MNIST/CelebA, super-résolution avec SRGAN (générateur ResNet + discriminateur PatchGAN), data augmentation synthétique pour dataset médical, métriques : FID (pytorch-fid), SSIM, PSNR
Vision Transformers (ViT) — PyTorch & HuggingFace	Self-Attention et Multi-Head Attention (nn.MultiheadAttention), patch embedding (nn.Unfold ou einops), positional encoding, ViT original, Swin Transformer (shifted windows), chargement via timm ou Hugging Face transformers
TP ViT — Segmentation (HuggingFace + PyTorch)	Fine-tuning ViT/Swin via timm sur dataset custom, segmentation sémantique avec SegFormer (Hugging Face), comparaison CNN vs ViT (accuracy, vitesse, données requises), visualisation attention maps

Détail des Modules — Jour 4

Module 4.1 : Generative Adversarial Networks (GAN) en PyTorch

L'implémentation PyTorch d'un GAN nécessite une boucle d'entraînement spécifique : le discriminateur et le générateur sont mis à jour en alternance via deux backward() distincts. La gestion des labels réels/faux et le detach() du générateur sont critiques.

Boucle d'entraînement PyTorch GAN :

- Mise à jour du discriminateur : real_loss + fake_loss avec fake.detach()
- Mise à jour du générateur : generator_loss avec les labels 'réels' pour tromper D

- DCGAN : nn.ConvTranspose2d dans G, nn.Conv2d avec stride=2 dans D
- cGAN : embedding des labels (nn.Embedding) concaténé à z dans G et à x dans D
- Pix2Pix : U-Net generator + PatchGAN discriminator (patches 70x70), loss L1 + adversariale
- WGAN : RMSprop, weight clipping ou gradient penalty (WGAN-GP)

Use Cases Computer Vision :

- Super-résolution d'images médicales (SRGAN avec perceptual loss VGG)
- Synthèse de données médicales pour augmentation de dataset (IRM, dermato)
- Traduction de domaine : IRM → CT-scan (CycleGAN, deux GAN couplés)
- Restauration et inpainting d'images dégradées
- Débruitage de signaux radar et sonar

Module 4.2 : Vision Transformers (ViT) — PyTorch, timm & Hugging Face

En PyTorch, les ViT sont accessibles via la bibliothèque timm (des centaines de modèles pré-entraînés) et Hugging Face Transformers. Le mécanisme d'attention est implémenté nativement via nn.MultiheadAttention depuis PyTorch 1.9.

APIs PyTorch & Écosystème :

- nn.MultiheadAttention(embed_dim, num_heads) : attention native PyTorch
- timm : timm.create_model('vit_base_patch16_224', pretrained=True, num_classes=N)
- timm : timm.create_model('swin_base_patch4_window7_224', pretrained=True)
- Hugging Face : SegformerForSemanticSegmentation.from_pretrained('nvidia/segformer-b2-finetuned-ade-512-512')
- einops library : rearrange(x, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)') pour patch embedding

Use Cases :

- Classification fine-grained : espèces biologiques, types de lésions cutanées (ISIC)
- Segmentation sémantique médicale : organes, tumeurs, rétine (SegFormer)
- Visualisation des attention maps : interprétabilité DINO (self-supervised ViT)
- Benchmark comparatif CNN (ResNet) vs ViT selon taille du dataset

Module 4.3 : Projet Fil Rouge

Chaque équipe de 2–3 participants réalise un pipeline DL complet sur un use case réel. Le projet est initié dès le Jour 1 et progressivement enrichi.

Thèmes proposés :

- Computer Vision médicale : classification/segmentation de pathologies (IRM cérébrale, rétinopathie diabétique)
- Contrôle qualité industriel : détection et localisation de défauts par CNN + CAM
- Traitement du signal biomédical : classification d'arythmies ECG avec LSTM + CNN
- Super-résolution satellite : SRGAN ou Diffusion Model sur images Sentinel-2
- Segmentation sémantique de scènes urbaines : SegFormer sur CityScapes
- Génération d'images médicales par Diffusion Models : DDPM conditionnel sur IRM/CT
- Interprétabilité XAI complète : pipeline Grad-CAM + SHAP + rapport d'audit sur modèle de classification médicale

Critères d'évaluation :

- Pertinence du choix architectural et justification
- Qualité du pipeline (prétraitement, augmentation, validation croisée)
- Performance des métriques (accuracy, mAP, IoU, SSIM selon le use case)
- Clarté de la présentation et démo live

6. Use Cases par Architecture

6.1 Computer Vision — Tâches Clés

Tâche	Architecture	Métriques	Exemples d'application
Classification	CNN (ResNet, EfficientNet) ViT, Swin, DeiT Transfer Learning	Accuracy, F1, AUC-ROC Top-5 Accuracy	Diagnostic médical, contrôle qualité, reconnaissance faciale
Segmentation Sémantique	U-Net, SegFormer DeepLab v3+, FCN	IoU (mIoU), Dice Pixel Accuracy	Segmentation d'organes (IRM/CT), routes, bâtiments (satellite)
Segmentation d'instance	Mask R-CNN YOLAct	AP, mAP, Dice	Comptage cellulaire, inspection PCB, détection de foules
Débruitage	ConvAE, U-Net DnCNN, VAE	PSNR, SSIM MSE	IRM à faible dose, images satellite, photographies nocturnes
Super-Résolution	SRGAN, ESRGAN RRDB	PSNR, SSIM FID	Images médicales, satellite, vidéosurveillance
Génération & Synthèse	GAN, DCGAN VAE, cVAE	FID, IS LPIPS	Augmentation de données médicales, synthèse de textures
Traduction d'image	Pix2Pix, CycleGAN cGAN	FID, SSIM	IRM→CT, cartes→photos satellite, colorisation
Génération par Diffusion	DDPM, DDIM Latent Diffusion, SR3	FID, IS PSNR, SSIM	Synthèse d'images médicales, super-résolution, inpainting, débruitage
Interprétabilité (XAI)	Grad-CAM, SHAP LIME, Captum, Attention Maps	Fidélité, Sparsité Plausibilité	Diagnostic médical, audit réglementaire, debugging de modèles, détection de biais

6.2 Traitement du Signal

Tâche Signal	Architecture	Métriques	Applications
Classification Temporelle	LSTM, GRU, BiLSTM 1D CNN, TCN, MLP	Accuracy, F1 Confusion Matrix	Classification ECG, HAR, fautes machines
Débruitage Signal	Denoising AE LSTM Seq2Seq, 1D U-Net	SNR, MSE Pearson r	ECG clinique, audio, signaux vibratoires
Détection d'anomalies	Autoencoder (reconstruction error) LSTM Forecast	AUC-ROC, F1 Precision/Recall	Maintenance prédictive, détection de fraude, IoT
Prédiction / Prévision	LSTM, GRU, Transformer 1D CNN	MAE, RMSE MAPE	Consommation énergétique, débit réseau, vibrations

Tâche Signal	Architecture	Métriques	Applications
Génération de signaux	VAE, GAN conditionnel RNN génératif	SNR, FID DTW	ECG synthétique, augmentation données capteurs

7. Ressources & Environnement Technique

Stack Technique — PyTorch

- Python 3.10+, Conda ou venv
- PyTorch 2.x (torch, torchvision, torchaudio) — framework principal
- torchvision : datasets, models (ResNet, EfficientNet, VGG...), transforms
- torchaudio : traitement du signal audio, spectrogrammes, STFT
- timm (PyTorch Image Models) : zoo de modèles ViT, Swin, EfficientNet...
- Hugging Face Transformers : ViT, SegFormer, DINO, fine-tuning simplifié
- torchmetrics : métriques ML (Accuracy, F1Score, AUROC, IoU, PSNR, SSIM)
- einops : manipulation élégante de tenseurs et patch embedding
- OpenCV, Albumentations : augmentation avancée d'images
- pytorch-fid : calcul du Fréchet Inception Distance pour évaluation des GAN
- diffusers (Hugging Face) : DDPMPipeline, DDPMScheduler, UNet2DModel pour modèles de diffusion
- Captum (PyTorch) : bibliothèque officielle d'interprétabilité — IntegratedGradients, DeepLift, GradientShap
- pytorch-grad-cam : Grad-CAM, HiResCAM, EigenCAM, FullGrad pour CNNs et Vision Transformers
- Weights & Biases (W&B) ou TensorBoard : suivi des expériences et hyperparamètres
- Jupyter Notebook / JupyterLab, VS Code avec extension PyTorch
- GPU : NVIDIA CUDA 11.8+ / 12.x, Google Colab Pro (T4/A100) ou cluster local

Ressources Pédagogiques

Livres de référence :

- Deep Learning with PyTorch — Eli Stevens, Luca Antiga, Thomas Viehmann (Manning, 2020)
- Programming PyTorch for Deep Learning — Ian Pointer (O'Reilly)
- Deep Learning — Ian Goodfellow, Yoshua Bengio, Aaron Courville (MIT Press)
- Computer Vision: Algorithms and Applications — Richard Szeliski (Springer, 2nd Ed.)
- Understanding Deep Learning — Simon J.D. Prince (MIT Press, 2023) — chapitres Diffusion Models & Interpretability

Documentation officielle :

- PyTorch Tutorials officiels : pytorch.org/tutorials — référence incontournable
- timm documentation : timm.fast.ai — zoo de modèles PyTorch
- Hugging Face Docs : huggingface.co/docs — Transformers, Datasets, Trainer API
- torchvision : pytorch.org/vision — datasets, models, transforms

Cours en ligne PyTorch :

- CS231n Stanford : Convolutional Neural Networks for Visual Recognition (PyTorch labs)
- fast.ai : Practical Deep Learning for Coders (100% PyTorch)
- PyTorch Lightning : formation structurée avec abstraction de la boucle d'entraînement

- Zero to Mastery : PyTorch for Deep Learning (2023, code-first)

Datasets utilisés en formation :

- ImageNet, CIFAR-10/100, Oxford-IIIT Pets (classification)
- CityScapes, ADE20K, PASCAL VOC (segmentation)
- PhysioNet MIT-BIH (ECG), UCI HAR (accéléromètre)
- MNIST, CelebA, FFHQ (génération)
- Chest X-Ray, ISIC Dermoscopy (médical)

Déploiement PyTorch

La formation intègre une introduction au déploiement de modèles PyTorch en production lors du Jour 4 :

- TorchScript : torch.jit.script() et torch.jit.trace() pour compiler le modèle (C++ runtime)
- ONNX Export : torch.onnx.export() pour interopérabilité avec TensorRT, OpenVINO, CoreML
- PyTorch Mobile : export pour iOS/Android via torch.utils.mobile_optimizer
- Quantification : torch.quantization pour modèles légers (int8 post-training)